/Collections/

Flora Belas

/O que é/

- O **Java Collections Framework (JCF)** é um conjunto dedicado de classes e interfaces localizado no pacote java.util que serve para armazenar, organizar e manipular grupos de dados como uma única unidade.
- Em vez de usar arrays tradicionais, que possuem tamanho fixo e exigem controle manual de posições, as coleções oferecem estruturas de dados dinâmicas e prontas para uso.

/O que é/

- **Collections** é o conjunto de estruturas do Java pra guardar vários valores
- Elas **crescem e diminuem sozinhas**. Sem tamanho fixo, sem contador
- Cada uma responde a uma pergunta diferente:

"me dá o que está na posição 2"	List
"me dá o valor guardado sob essa chave"	Map
"esse valor já está aqui?"	Set
"quem é o próximo da fila?"	Queue

/List/

- List é uma sequência em que cada item tem uma posição, começando em 0. Os itens ficam na ordem em que foram colocados e podem se repetir. Na prática, use sempre **ArrayList**.
- **Onde aparece numa empresa:** itens de um pedido, carrinho de compras, histórico de transações de uma conta (em ordem), lista de funcionários para um relatório, linhas lidas de uma planilha.

/Array vs List/

	Array	ArrayList
Tamanho	fixo	cresce
Criar	new int[5]	new ArrayList<>()
Adicionar	a[0] = 8	lista.add(8)
Acessar	a[0]	lista.get(0)
Quantidade	a.length	lista.size()
Imprimir direto	endereço	o conteúdo

/ArrayList/

- É a **lista mais usada do Java**. **array que cresce**
- Precisa do import java.util.ArrayList; no topo
- O <String> entre os sinais diz **o que a lista guarda**
- Os parênteses vazios no new criam ela vazia
- Não precisa dizer o tamanho. Não precisa de contador.
- Não recebem tipos primitivos (apenas Integer, String, Boolean, Double...)

```
import java.util.ArrayList;

ArrayList<String> nomes = new ArrayList<>();

nomes.add("Ana");
nomes.add("Bia");
nomes.add("Carla");

System.out.println(nomes);
```

/ArrayList/

Método	O que faz
add("Ana")	adiciona no fim da lista
get(0)	pega pela posição
size()	tamanho
remove(1)	remove pela posição
set(0, "Zoe")	troca o valor da posição
contains("Bia")	está na lista?
indexOf("Carla")	em que posição está?
isEmpty()	está vazia?
lista.addAll(List.of(“janine", "ana"));	adiciona vários valores

/Hashmap/

Um HashMap em Java é uma estrutura de dados que armazena informações em pares de chave e valor, funcionando de forma parecida com um dicionário

- **Não ordenado:** Os elementos não ficam salvos em uma ordem específica de inserção.
- **Permite nulos:** Aceita uma única chave null e múltiplos valores null.
- **Performance:** As operações básicas de adicionar (put) e buscar (get) são extremamente rápidas.

/Hashmap/

- HashMap guarda pares: **uma chave e um valor**
- Você não busca por posição, busca **pela chave**
- É um **dicionário**: você procura a palavra e acha o significado
- Ou uma agenda: procura o nome e acha o telefone
- Declara **dois tipos**: o da chave e o do valor

/HashMap: Como Funciona/

- **Chave (Key):** É um identificador único. Você usa a chave para encontrar o valor correspondente de forma rápida. Não pode haver chaves duplicadas.
- **Valor (Value):** É o dado associado àquela chave. Os valores podem se repetir.
- **Tabela Hash:** Internamente, ele usa uma técnica chamada hashing para calcular a posição exata onde o dado está guardado, o que deixa a busca muito rápida.

/HashMap: Código/

```
import java.util.HashMap;

HashMap<String, Integer>idades = new HashMap<>();
idades.put("Ana", 25);
idades.put("Carlos", 30);
int idadeAna = idades.get("Ana");
```

/HashMap: Código/

Método	O que faz
put("Ana", 28)	guarda o par
get("Ana")	busca pela chave
getOrDefault("Zoe", 0)	busca, e devolve algo se não achar
containsKey("Ana")	essa chave existe?
containsValue(28)	esse valor existe?
remove("Ana")	remove o par
size()	quantos pares tem
isEmpty()	está vazio?
keySet()	mostra todas as chaves
values()	mostra todos os valores